

Project Title: Car Service & Maintenance Reminder App (Automated Testing using Appium & Java)

1. Introduction

Mobile applications play a crucial role in managing vehicle maintenance and service schedules. The **Car Service & Maintenance Reminder App** (Drivvo) helps users track essential activities such as oil changes, tire replacements, insurance renewals, and periodic maintenance.

Manual testing of such applications is repetitive, time-consuming, and error-prone. To overcome these challenges, this project focuses on **automated mobile application testing** using **Appium with Java** on a real Android device. Automation improves test execution speed, accuracy, reliability, and regression coverage.

2. Problem Statement

Challenges Identified

- Repetitive manual test execution
- High probability of human error
- Difficulty in performing frequent regression testing
- Validation of dynamic UI elements such as date pickers and scrollable views
- Need to ensure accuracy in reminder creation, modification, deletion, and report generation

Proposed Solution

Implement a **mobile automation framework using Appium and Java** to ensure consistent, repeatable, and reliable validation of core workflows in the application.

3. Objectives of the Project

- Automate key functionalities of the car maintenance reminder app
- Reduce manual testing effort and execution time
- Improve test accuracy and reliability
- Enable effective regression testing

- Generate detailed execution reports for analysis
-

4. Tools & Technologies Used

Tool / Technology	Purpose
Java	Programming language for automation scripts
Appium (UiAutomator2)	Mobile automation testing tool
Android	Target mobile platform
Eclipse IDE	Development environment
Appium Server	Middleware for test execution
HTML	Test report generation

5. Mobile Automation Architecture

The automation framework follows a **Client–Server Architecture**:

- **Automation Script (Java)**: Contains test logic and sends commands
- **Appium Server**: Converts commands using WebDriver protocol
- **UiAutomator2 Driver**: Communicates with Android OS
- **Android Device**: Executes the Drivvo application

This architecture ensures stable and scalable mobile automation testing.

6. Execution Flow

1. Automation script initiates test execution
 2. Appium Server receives commands
 3. UiAutomator2 interacts with the Android device
 4. Application actions are executed
 5. Results are validated and logged
-

7. Test Case Development & Execution

Test cases were designed to reflect **real-world user scenarios** and ensure comprehensive coverage of application functionality.

Test Case 1: Add Reminder (Oil Change)

- Objective: Verify successful reminder creation
- Expected Result: Reminder is added and displayed correctly

Test Case 2: Edit Reminder

- Objective: Verify reminder modification
- Expected Result: Updated details are saved successfully

Test Case 3: Delete Reminder

- Objective: Verify reminder removal
 - Expected Result: Reminder is deleted from the list
-

8. Automation Configuration

Driver Capabilities

- Platform Name: Android
- Automation Name: UiAutomator2
- Device Name: Real Android Device
- App Package: br.com.ctncardoso.ctncar
- App Activity: StartActivity
- NoReset: True
- NewCommandTimeout: 60 seconds

Additional Features

- Scroll handling using UiScrollable
 - Date picker automation
 - Exception handling with try-catch blocks
 - Screenshot capture on failures
 - HTML report generation
-

9. Validation & Reporting

Validation Techniques

- UI element verification
- Execution flow validation
- Console log monitoring

Reporting Mechanism

- Automated HTML reports
 - Pass/Fail status summary
 - Screenshots on failure
 - Detailed execution logs
-

10. Testing Methodologies Used

- Functional Testing
- Automation Testing
- Regression Testing
- End-to-End Testing
- UI Testing

These methodologies ensure comprehensive coverage and high-quality validation.

11. Traceability Matrix

A traceability matrix was maintained to map **application requirements to test cases**, ensuring complete functional coverage and transparency in testing.

12. Challenges Faced & Solutions

Challenges

- Dynamic XPath elements
- Date picker automation
- Scroll functionality
- Synchronization issues
- Real device connectivity

Solutions

- Dynamic XPath strategies with waits

- Custom date picker utilities
 - UiScrollable integration
 - Controlled wait mechanisms
 - Robust exception handling
-

13. Limitations

- Use of Thread.sleep() instead of explicit waits
 - No Page Object Model implementation
 - No TestNG framework integration
 - Sequential test execution only
 - Hardcoded locators and test data
-

14. Future Enhancements

- Implement Page Object Model (POM)
 - Integrate TestNG framework
 - Add Extent Reports
 - Replace fixed waits with explicit waits
 - Enable parallel testing
 - CI/CD integration using Jenkins
-

15. Conclusion

This project successfully demonstrates **mobile automation testing** for a car maintenance reminder application using **Appium and Java** on a real Android device. Automated workflows, structured reporting, and real-device execution significantly reduce manual effort while improving reliability and test coverage.

The framework provides a strong foundation for scalable and maintainable mobile test automation.